Laboratorio2 Procesamiento de ficheros de texto

Uno de los puntos fuertes de los sistemas Unix estriba en la facilidad con la que pueden analizar los ficheros de texto en ellos. Estos sistemas incluyen una serie de herramientas que permiten realizar una gran cantidad de manipulaciones en estos ficheros sin necesidad de instalar ninguna herramienta especializada. Estas son las herramientas que vamos a presentar en esta sección.

Ficheros de texto y binarios

Antes de comenzar a analizar los este tipo de ficheros debemos aclarar qué es y qué no es un fichero de texto. Un fichero de texto es un fichero dividido en líneas y cuyo contenido es texto. A pesar de lo que pudiese parecer a priori, un documento del Microsoft Office o del LibreOffice no es un fichero de texto. La información contenida en estos documentos es binaria y sólo los programas especialmente creados para abrir estos ficheros pueden acceder a ella de un modo inteligible. En un documento como en un fichero *doc* además de texto se guarda la información sobre el formato, imágenes, tablas, etc.. Por el contrario en un fichero de texto sólo hay caracteres alfanuméricos (letras y números), retornos de carro y tabuladores.

Los ficheros de texto pueden ser abiertos e inspeccionados sin necesidad de hacer uso de un software especial diseñado para trabajar con ellos. Si nos llegase un documento del LibreOffice y no tenemos el programa instalado no podríamos poder verlo, pero si nos envían un fichero de texto lo podremos ver y editar con las herramientas que vienen instaladas por defecto en el sistema operativo.

Uno de los puntos fundamentales de la filosofía Unix, es la utilización de ficheros de texto. Mientras otros sistemas operativos favorecen la utilización de ficheros binarios, que deben ser acompañados de herramientas especiales para poder manipularlos, en Unix se optó por crear un conjunto de herramientas para manipulación de ficheros de texto y por utilizar para los ficheros del sistema sólo ficheros de texto siempre que esto fuese posible. Estas herramientas de manejo de ficheros de texto nos permiten realizar complejas manipulaciones de un modo muy sencillo y son uno de los principales atractivos de los sistemas Unix para el manejo de grandes cantidades de información.

Imprimiendo ficheros *cat*, *head* y *tail*

En esta sección vamos a trabajar con el archivo de prueba `microarray_adenoma_hk69.csv`. En este fichero están almacenados los resultados de un experimento de expresión diferencial en el que se han analizado distintos adenomas. Este es un fichero tabular en el que la información que se representa dividiendo los campos mediante comas o tabuladores. Estos ficheros que representan tablas en formato texto se se denominan *csv* (*Comma separated values*). En este caso cada fila del fichero corresponde a una sonda del microarray y cada columna a una propiedad sobre la sonda o sobre el resultado de la hibridación sobre ella.

Lo primero que podemos hacer con un fichero de texto es abrirlo para ver sus contenidos. Existen editores de texto que funcionan en ventanas, como el *gedit*, y editores que funcionan en la terminal, como el *nano*. Por desgracia a veces los ficheros con los que vamos a trabajar son tan grandes que incluso los buenos editores de texto pueden tener problemas para abrirlos.

Otra forma de acceder a los contenidos del fichero sería imprimir el fichero en la terminal utilizando el comando *cat*:

```
~$ cat microarray_adenoma_hk69.csv
```

Pero si intentamos hacerlo la terminal quedará bloqueada durante bastante tiempo puesto que el fichero es muy grande. Por cierto, si hemos ejecutado el comando anterior y ahora queremos terminar (o matar) el programa que está corriendo en la consola (en este caso el *cat*) podemos utilizar la combinación de teclas *control* + *c*. Esto suele hacer que los programas terminen lo que estén haciendo inmediatamente, se apaguen y vuelva a mostrarse el *prompt*.

Para hacernos una idea del contenido del fichero sin bloquear la terminal podemos imprimir tan solo las primeras líneas utilizando el comando *head*:

```
~$ head microarray_adenoma_hk69.csv
"!Exptid=10029"
"!Experiment Name=Adenoma (HK69)"
"!Organism=Homo sapiens"
"!Category=Adenoma"
"!Subcategory=Liver"
"!Experimenter=Xin Chen"
"!Contact email=chenx@pharmacy.ucsf.edu"
"!Contact Address1=Dept. of Biopharmaceutical Sciences"
"!Contact Address2=513 Parnassus Ave. S-816"
"!Contact Address3=Box 0446"
```

Existe otro comando equivalente pero que nos permite imprimir el final de los archivos (*tail*):

```
~$ tail microarray_adenoma_hk69.csv
24183       "EMPTY" "EMPTY" 19      27      32      0
```

Tanto a *head* como a *tail* podemos pedirles que impriman el número de líneas que nosotros deseemos:

```
~$ head -n 2 microarray_adenoma_hk69.csv
"!Exptid=10029"
"!Experiment Name=Adenoma (HK69)"
```

Navegando por un fichero de texto

En muchas ocasiones para familiarizarnos con el fichero lo mejor es abrirlo y navegar por él. Podríamos abrir el fichero con un editor de texto, pero ya hemos visto que si es muy grande podríamos tener problemas. Existe una herramienta capaz de abrir ficheros de texto inmensos sin problemas, *less*. *less* es un visor de ficheros de texto, con este programa no podremos editar el fichero, pero sí navegar por su contenido. *less* es un

programa interactivo por lo que cuando lo ejecutemos se abrirá ocupando el terminal y haciendo desaparecer el *prompt*. En cualquier momento podemos salir de *less* pulsando la tecla “q”.

```
~$ less microarray_adenoma_hk69.csv
```

Dentro de *less* disponemos de varios comandos para movernos por el fichero:

```
* barra de espacio: página siguiente.
* b: página anterior.
* 100g: va a la línea 100 (o la que le indiquemos)
* -S: corta o no corta las líneas largas
* /palabra: Busca la cadena de texto que le indiquemos (acepta
expresiones regulares)
* n: va a la siguiente palabra que coincide con la búsqueda
* N: va la palabra anterior que coincide con la búsqueda
* q: sale del programa
```

grep

Una tarea que vamos a tener que realizar habitualmente es la de seleccionar diversas líneas en un fichero de texto. Por ejemplo, imaginemos que queremos saber cual es la expresión de los genes relacionados con la leucemia en el fichero del *microarray*. Esto, que en otros sistemas operativos podría resultar bastante complejo de hacer en Unix es trivial:

```
~$ grep leukemia microarray_adenoma_hk69.csv
3       "IMAGE:302190"  "MLL"   "Myeloid/lymphoid or mixed-lineage
leukemia (trithorax homolog, Drosophila)"
```

grep (*Generalized Regular Expression Parser*) toma un fichero de entrada (o el *standard input*) y filtra las líneas que contienen el patrón de búsqueda que le hemos dado. Por defecto incluye en el resultado las líneas que contienen el patrón, pero podríamos pedirle que haga lo contrario utilizando la opción *v* (*inVert*).

grep diferencia entre mayúsculas y minúsculas, pero podemos cambiar este comportamiento con la opción *i* (ignore case):

```
~$ grep -i leukemia microarray_adenoma_hk69.csv
3       "IMAGE:302190"  "MLL"   "Myeloid/lymphoid or mixed-lineage
leukemia (trithorax homolog, Drosophila)"
```

grep puede utilizarse para buscar recursivamente en todos los ficheros contenidos en un directorio. Por ejemplo podríamos buscar la palabra *leukemia* en todos los ficheros presentes en alguno de nuestros directorios:

```
~$ grep -r  leukemia ~
```

Si queremos saber en que posiciones del fichero original estaban las líneas que hemos encontrado podemos pedirle que imprima dichos números de línea:

```
~$ grep -n leukemia microarray_adenoma_hk69.csv
```

grep tiene otras muchas opciones útiles, en su manual están todas descritas:

```
~$ man grep
```

grep hace uso de las expresiones regulares que veremos posteriormente por lo que su potencia es mucho mayor que la que ahora podemos imaginar.

Otra de las ventajas de *grep* viene dada por su adherencia a los principios Unix lo que implica que podemos utilizar *grep* enlazándolo con otros programas. Imaginemos que queremos hacer una búsqueda limitada a las primeras cien líneas del fichero, podríamos hacer:

```
~$ head -n 100 microarray_adenoma_hk69.csv | grep leukemia
```

Lo que hemos hecho es utilizar el comando *head* para leer las primeras cien líneas del archivo, pero en vez de imprimir el resultado en pantalla hemos redirigido su salida mediante una tubería (*pipe*) al comando *grep*. Éste ha tomado estas cien líneas y las ha filtrado con la palabra *leukemia*. Podríamos además redirigir el resultado final a un fichero para guardar el resultado:

```
~$ head -n 100 microarray_adenoma_hk69.csv | grep leukemia >
busqueda_leukemia_100.txt
```

Todos los comandos de manejo de texto que vamos a ver en esta sección tienen esta capacidad. Todos siguen el principio de realizar una tarea concreta pero teniendo la capacidad de enlazarse con otros.

wc

wc (*Word Count*) sirve para contar líneas, palabras y caracteres. Por ejemplo podemos enlazarlo con el *grep* y contar cuantos genes relacionados con la leucemia hay en el fichero.

```
~$ grep leukemia microarray_adenoma_hk69.csv | wc
     89    5256   31600
```

cut

Cuando el fichero está dividido en campos, como en el caso de la tabla que estamos utilizando, podemos seleccionar alguno de estos campos utilizando el comando *cut*. Por ejemplo mejoremos la búsqueda que habíamos hecho quedándonos tan solo con el nombre y la descripción del gen:

```
~$ grep leukemia microarray_adenoma_hk69.csv | cut -f 3,4
"BAALC" "Brain and acute leukemia, cytoplasmic"
"DEK"   "DEK oncogene (DNA binding)"
```

Con el parámetro *-f* le indicamos la lista de campos (*fields*) que queremos seleccionar.

cut asume que los campos en el fichero están divididos por tabuladores. Pero podríamos indicarle que los campos están divididos de otro modo, por ejemplo por comas:

```
~$ cut -d ',' fichero_separado_por_comas.txt
```

Para tener una idea completa de las capacidades ofrecidas por el comando conviene consultar su manual:

```
~$ man cut
```

sed

La lista de genes que hemos obtenido en el apartado anterior puede servirnos para muchos propósitos, pero todavía no está limpia del todo, quedaría mejor si eliminásemos las comillas que rodean los campos. Esta tarea podemos realizarla con el comando *sed* (*stream editor*). *sed* toma las líneas de una en una, les aplica la transformación que le indiquemos y devuelve las líneas modificadas. Por ejemplo, eliminemos las comillas:

```
~$ grep leukemia microarray_adenoma_hk69.csv | cut -f 3,4 | sed
"s/\"//g"
BAALC   Brain and acute leukemia, cytoplasmic
DEK     DEK oncogene (DNA binding)
```

La sintaxis utilizada por *sed* puede resultar algo oscura al principio, pero un mínimo conocimiento de este comando nos permitirá hacer modificaciones en el texto que de otro modo serían muy complejas. En este caso, el comando sed que hemos utilizado es “s/”//g”. En primer lugar hemos indicado a *sed* qué queríamos hacer. Podemos, como es el caso, substituir un patrón por otro (comando s, sustituir), pero también podríamos pedirle que eliminase líneas (comando d, delete). Para substituir hay que indicarle qué queremos substituir y por qué:

```
s/patrón_a_substituir/nuevo_patrón/
```

En el ejemplo hemos substituido las comillas por nada. En principio deberíamos haber escrito “s/”//”. Pero dado que las comillas tienen un significado especial en este contexto debemos *escaparlas*, por eso escribimos “s/"//”. Además, a la expresión le hemos añadido una “g”, con este modificador le indicamos a *sed* que no queremos que substituya sólo la primera aparición del patrón en la línea sino todos los que haya.

sed acepta expresiones regulares como patrones y con ellas puede realizar prácticamente cualquier substitución que podamos imaginar.

sort

Si deseamos ordenar alfabéticamente un fichero de texto tan sólo tenemos que utilizar el comando *sort*. Por ejemplo, podemos ordenar los genes relacionados con la leucemia:

```
~$ grep leukemia microarray_adenoma_hk69.csv | cut -f 3 ,4|
sort
```

sort también puede hacer ordenaciones numéricas y puede ordenar por cualquiera de los campos presentes en el fichero. Para ver qué se puede hacer con *sort* lo más recomendable es leer su manual.

uniq

En el ejemplo anterior, al ordenar con *sort*, hemos visto que en la lista que hemos obtenido hay genes repetidos. Con el comando *uniq* podemos eliminar las líneas duplicadas consecutivas. Para que la eliminación sea completa hay que recordad ordenar con *sort* antes de utilizar *uniq*:

```
~$ grep leukemia microarray_adenoma_hk69.csv | cut -f 3 ,4|
sort | uniq
```

paste

paste une ficheros tabulares línea por línea. Supongamos que tenemos dos ficheros, uno con datos sobre la progresión de la enfermedad de una serie de enfermos y otro con el genotipado de los mismos:

```
pacientes:
id_paciente,nivel_colesterol
1,190
2,250
3,220
4,260
5,160

genotipado:
id_paciente,SNP_a,SNP_b
1,AA,CC
2,AC,GG
3,AA,CG
4,AT,GG
5,AA,CC
```

Podemos fusionar los dos archivos usando el comando paste:

```
~$ paste -d',' pacientes.txt genotipado.txt
id_paciente,nivel_colesterol,id_paciente,SNP_a,SNP_b
1,190,1,AA,CC
2,250,2,AC,GG
3,220,3,AA,CG
4,260,4,AT,GG
5,160,5,AA,CC
```

join

join es una herramienta muy potente, pero sencilla. Permite unir dos ficheros de texto tabulares en uno usando una columna como clave común. *join* es una especie de *paste* en la que las filas no tienen porqué estar ordenadas y la columna común que sirve como enlace entre ambas tablas no queda duplicada. Imaginemos que tenemos dos ficheros,

en uno se describen los datos de colecta de una serie de variedades de tomate y en otro la caracterización de los mismos:

```
pasaporte:
nombre        origen
valenciano    Valencia
penjar        Barcelona
amarillo      Roma
industria     Murcia

caracterización:
nombre      color     forma            duración
valenciano  rojo      apuntado         corta
penjar      rojo      ovalado          larga
amarillo    amarillo  ovalado grande   normal
industria   rojo      ovalado          normal
```

join nos permite unir estas dos tablas en una sola utilizando el nombre de la variedad de tomate como la clave de unión:

```
$ join tomate_pasaporte.txt tomate_caracterizacion.txt
nombre     origen    color     forma           duración
valenciano Valencia  rojo      apuntado        corta
penjar     Barcelona rojo      ovalado         larga
amarillo   Roma      amarillo  ovalado grande  normal
industria  Murcia    rojo      ovalado         normal
```

Codificación de caracteres

Los ordenadores codifican los caracteres del lenguaje natural utilizando números. Los ficheros de texto no son pues más que ficheros de números que el ordenador transforma en caracteres utilizando una tabla de codificación de caracteres antes de imprimirlos. Una de las tablas de codificación más populares es la ASCII, en ella se incluyen los caracteres utilizados en la lengua inglesa, pero no los acentos o los caracteres de otros alfabetos. Para solucionar el problema el ASCII original fue ampliado creándose distintas tablas, una por cada lengua natural. De este modo con el ASCII correspondiente al castellano podemos crear ficheros de texto con acentos. Pero al crear distintas tablas de codificación surgió un problema, si nos equivocamos de tabla al decodificar el fichero obtendremos caracteres extraños que no se corresponden con lo que originalmente se había escrito. Para solucionar el problema se creó la norma *Unicode*, que es una gran tabla con unos 100713 símbolos que codifican prácticamente todos los caracteres utilizados en casi cualquier alfabeto humano.

El problema práctico al que nos enfrentamos es que para poder abrir un fichero de texto que incluya caracteres no ingleses debemos conocer la tabla de caracteres en la que fue codificado. En Linux los ficheros de texto se codifican mediante *UTF-8* que es una de las tablas del estándar *Unicode*, pero en *Windows* no se sigue este estándar. Las instalaciones de Windows en castellano utilizan la tabla Windows-1252.

Normalmente los editores de texto permiten elegir la tabla de caracteres en la que el fichero ha sido codificado. Si abrimos un fichero y vemos que tiene símbolos raros lo más normal es que estemos utilizando una tabla de caracteres equivocada. Este problema también suele ocurrir en las páginas web. Una página web no es más que un

fichero de texto con un formato determinado. Si el navegador no es capaz de inferir la codificación del fichero los acentos pueden aparecer como caracteres extraños.

En Linux hay varios programas que nos permiten cambiar los ficheros de una tabla de codificación a otra, uno de ellos es *iconv*. Por ejemplo si tenemos un archivo con una codificación UTF-8 o ISO-8859-1 podemos pasar de una a otra con *iconv*:

```
~$ iconv -t ISO-8859-1 -f UTF-8 sagan.iso-8859-1 >
sagan.utf-8_mod.txt
```

```
~$ iconv -f ISO-8859-1 -t UTF-8 sagan.utf-8.txt >
sagan.iso-8859-1_mod.txt
```

Fin de línea

Para marcar el final de una línea en un fichero de texto se utiliza un carácter especial. Por desgracia no ha habido un acuerdo a la hora de decidirse cual es el carácter que debe usarse. Unix, Windows y Mac OS utilizan caracteres diferentes. Afortunadamente los editores de texto suelen ser capaces de detectar las tres versiones y muestran el fichero con las líneas que esperamos. Desgraciadamente el notepad del Windows no es capaz de hacerlo por lo que si abrimos un fichero de texto creado en Linux en el notepad nos aparecerá como una larga línea con símbolos extraños intercalados. Casi todos los editores de texto permiten hacer el cambio entre los distintos finales de línea.

Ejercicios

Se ha realizado un estudio de un nuevo tratamiento para un linfoma y nos han enviado dos ficheros. En el llamando `cancer_progresion.txt` se encuentran tanto los datos de los pacientes como el resultado del tratamiento. En el segundo `fichero` (**cáncer_ciego.txt**), se encuentra la tabla que nos permitirá desentrañar el ensayo del doble ciego, con el identificador de cada paciente y la dosis de droga que se le administró.

1. ¿Cuántos pacientes había en el estudio?
2. ¿De cuántos pacientes no tenemos datos de progresión?
3. Convertir la separación de comas de la tabla de doble ciego a tabuladores. (El tabulador se escribe como \t)
4. Unir la tabla de los resultados de la terapia con la del doble ciego.
5. Transformar el fichero resultante las comas a tabuladores.
6. ¿Cómo les ha ido a los pacientes según el tipo de tratamiento? (Placebo está escrito con mayúsculas y minúsculas)
7. Crea un fichero con los primeros 100 resultados del microarray de adenoma que incluya sólo las primeras 10 columnas.
8. Ordena el fichero micro.txt generado en la cuestión 7 por el nombre del gen (campo 3) y por el id de la fila, pero en orden numérico reverso.
9. Disponemos de dos ficheros con secuencias de ADN (`seqs_1.fasta` y `seqs_2.fasta`). ¿Cuántas secuencias hay en cada fichero? ¿Hay alguna secuencia presente en ambos ficheros? (En los archivos de secuencia tipo fasta el nombre de las secuencias se encuentra en las líneas que comienzan por el símbolo >)
10. Disponemos de un fichero con secuencia de ADN (`seqs_3.fasta`), puedes extraer los nombres de las sequencias?
11. disponemos de un fichero con el resultado de un mapeo en formato SAM(`tomate.sam`). ¿cuantas secuencias se han mapeado? ¿ cuantas se han mapeado en dirección reversa(mirad la segunda columna: 0 forward; 16 reverse)? ¿ cuantos y cuales son los unigenes a los que se ha podido mapear alguna secuencia? Ordena las nombres de secuencias mapeadas con el orden del unigene y la posición en el unigene